ECA304 PROGRAMAÇÃO ORIENTADA A OBJETOS

Prof. Alexandre H. Moreira

Aula 07: Classes e Objetos II

Público, Protegido e Privado

- Considere o código a seguir:

```
class minhaClasse():
    def __init__(self):
        self.publico = "Eu sou publico."
        self._protegido = "Eu sou protegido."
        self.__privado = "Eu sou privado."

teste = minhaClasse()
print(teste.publico)
print(teste._protegido)
print(teste.__privado)
```

- E sua saída:

```
Eu sou publico.
Eu sou protegido.
Traceback (most recent call last):
  File "aula_09.py", line 17, in <module>
    print(teste.__privado)
AttributeError: 'minhaClasse' object has no attribute '__privado'
```

Público, Protegido e Privado – Resumindo

Nome	Tipo	Significado
nome	Público	Pode ser utilizado livremente dentro ou fora de uma definição de classe.
_nome	Protegido	Não deve ser utilizado fora da definição de classe, a menos que dentro de uma definição de subclasse.
__nome	Privado	Inacessível e invisível. Não é possível ler nem escrever nesses atributos, exceto dentro da própria definição de classe.

Métodos Setter e Getter

- Podemos definir os métodos getter e setter como:
  - Um método que permite acessar um atributo;
  - Um método que permite alterar o valor de um atributo.
- Considere o código a seguir:

```
class Pessoa():
    def_init_(self, nome):
        self.nome = nome
        self._idade = None
        self.__CPF = None
    def getIdade(self):
        if self._idade == None:
            print("Idade não definida!")
        return self._idade
    def setIdade(self, idade):
        self._idade = idade
```

Métodos Setter e Getter


```
    def getCPF(self):
        if self._CPF == None:
            print("CPF não definido!")
        return self.__CPF
    def setCPF(self, CPF):
        self.__CPF = CPF

joao = Pessoa("João da Silva")
print(joao.nome)
print(joao.getIdade())
joao.setIdade(24)
print(joao.getIdade())
joao.setCPF("123.456.789-00")
print(joao.getCPF())
```

```
Joao da Silva
Idade não definida!
None
24
123.456.789-00
```

PRIVADO PERO NO MUCHO!

- Acabamos de verificar que quando adicionamos "__" a algum atributo o tornamos privado, porém, é possível driblar isso:

```
Class minhaClasse(object):
    def __init__(self):
        self.publico = "Eu sou publico."
        self._protegido = "Eu sou protegido."
        self.__privado = "Eu sou privado."

teste = minhaClasse()
print(teste._minhaClasse__privado)
```

```
Eu sou privado.
```

- Não existem métodos ou atributos privados em Python!

Boas práticas

Note: Python doesn't have the notion of access modifiers, such as `private`, `protected`, and `public`, to restrict access to attributes and methods in a class. In Python, the distinction is between **public** and **non-public** class members.

If you want to signal that a given attribute or method is non-public, then you should use the well-established Python convention of prefixing the name with an underscore (_).

Note that this is just a convention. It doesn't stop you and other programmers from accessing the attributes using **dot notation**, as in `obj._attr`. However, it's bad practice to violate this convention.

Exercício para entrega

Caixa Eletrônico

Caixa Eletrônico

- Criar uma classe chamada “Cliente” que contenha os seguintes atributos:
  - nome (Público)
  - CPF (Privado)
  - saldo (Privado)
  - limite(Privado)
- Deve ser informado ao método construtor no momento de criação de uma instância o nome e o CPF. Deve-se inicializar o atributo saldo com “0”;

Caixa Eletrônico

- Devem ser criados os seguintes métodos:
  - exibeCPF();
  - exibeSaldo();
  - deposito(valor);
  - pagamento(valor);
- O método “deposito” acresce “valor” ao saldo;
- O método “pagamento” decresce “valor” ao saldo, caso o valor do “saldo” + “limite” seja suficiente, caso contrário deve ser exibida uma mensagem de erro;